Software Architecture and Design Specification (SAD)

Smart Traffic Complaint and Monitoring System

Project team: Arya, Ansh, Arnav, Avyukt

Technology baseline: MERN (MongoDB, Express.js, React, Node.js)

Document status: Prepared for submission review; test execution results must be filled only after tests are run.

1. Introduction

This document describes the proposed MERN architecture and design for the core Smart Traffic Complaint and Monitoring System. The design is a baseline for implementation; deviations should be documented.

2. Architecture Overview

2.1 Architecture pattern

The system uses a three-tier client-server architecture: React presentation layer, Node.js/Express application/API layer, and MongoDB persistence layer. The browser communicates with the API using HTTPS and JSON in deployment.

2.2 Components

Component Diagram — MERN Architecture

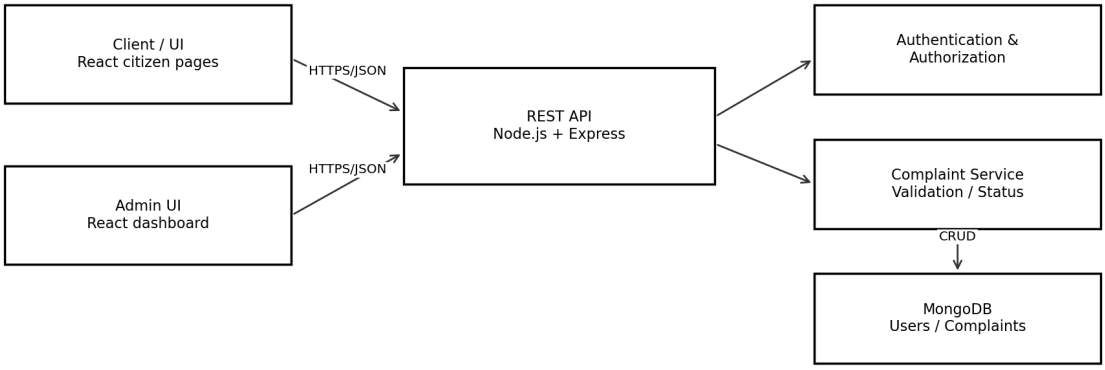


Figure 1. Component diagram for the MERN architecture.

- React citizen UI: registration/login, complaint submission, complaint list, status view.
- React administrator UI: complaint list, filters, priority/assignment/status management, dashboard.
- Express REST API: routes, request validation, authentication, authorization, and response formatting.
- Authentication/authorization middleware: validates identity and enforces citizen/admin permissions.
- Complaint service: business rules for create/list/filter/update operations.

- MongoDB: stores user accounts and complaint documents.

2.3 Data model

Entity	Key fields	Notes
User	_id, name, email, passwordHash, role, createdAt	email unique; role is citizen or admin
Complaint	_id, citizenId, category, description, locationDescription, status, priority, assignedTo, createdAt, updatedAt	citizenId references the submitting user; status/priority use enumerated values

2.4 Architecture security

- Hash passwords with a suitable salted password-hashing library; never store plaintext passwords.
- Validate authentication and authorization on the server for every protected route.
- Enforce complaint ownership when citizens request complaint details.
- Restrict administrator operations to accounts with the administrator role.
- Validate fields and allowed enum values; use safe MongoDB query construction.
- Use environment variables for secrets; do not commit secrets to the repository.
- Return generic error responses and log diagnostic detail only in protected server logs.

2.5 Requirement traceability

Component/design element	Requirements supported
React citizen UI	FR-01–07; NFR-02, NFR-04
React administrator UI	FR-08–13
Express API and complaint service	FR-03–14; NFR-01, NFR-06, NFR-07, NFR-10
Authentication/authorization middleware	FR-01, FR-02, FR-06, FR-15; SR-01, SR-02; NFR-08, NFR-09
MongoDB data model	FR-04–13; NFR-07; SR-03, SR-05
Validation and safe error handling	FR-14; NFR-06, NFR-10; SR-04–SR-06

3. Detailed Design

3.1 Sequence diagram: citizen submits complaint

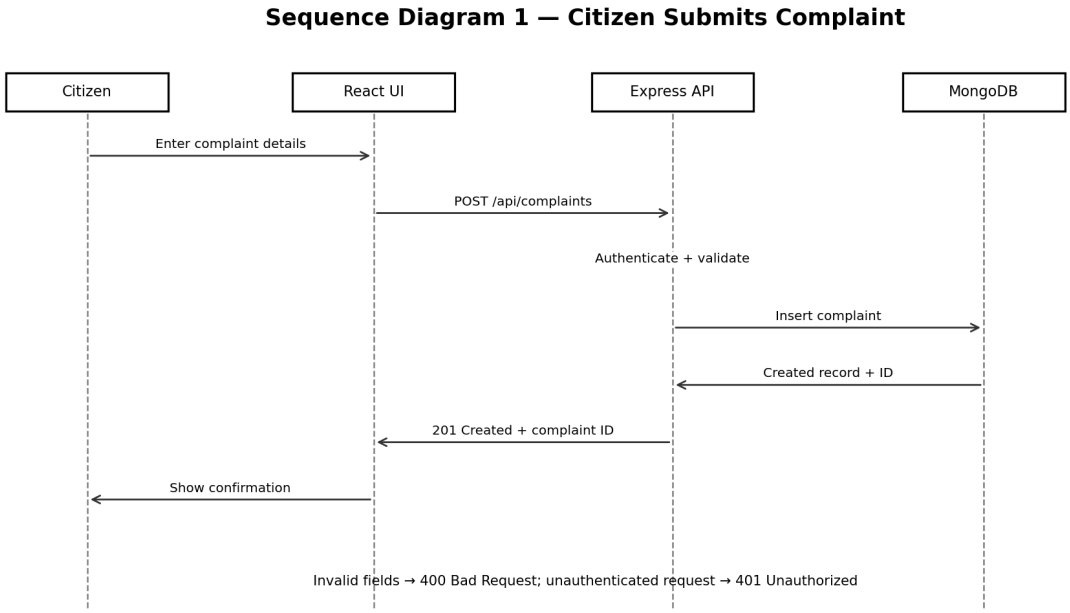


Figure 2. Submission flow: authenticate, validate, save, and return complaint identifier.

3.2 Sequence diagram: administrator updates status

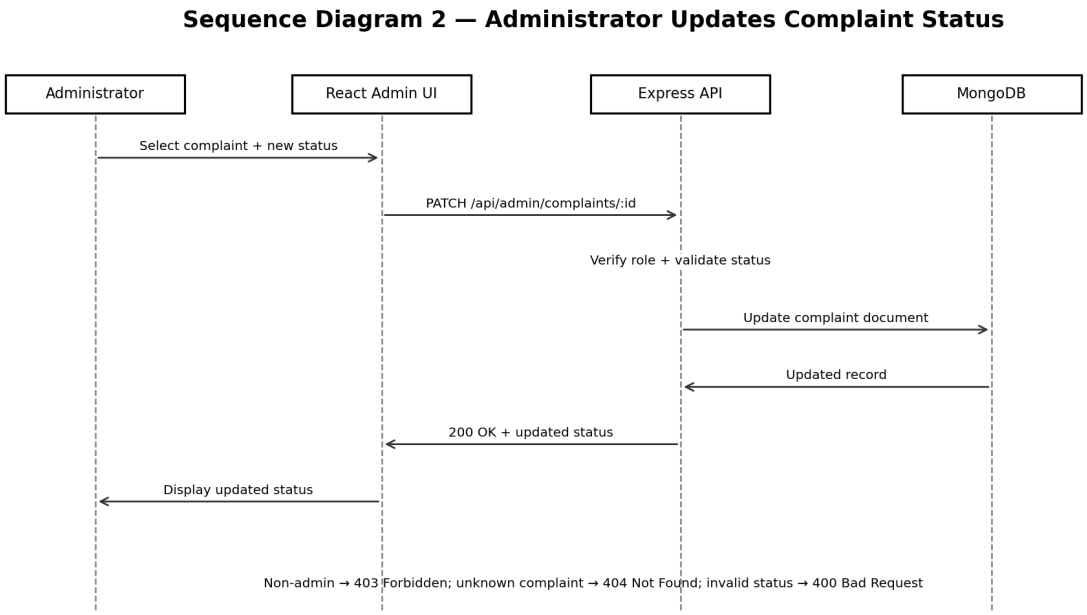


Figure 3. Admin status update flow with role validation.

3.3 API design

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Register user; returns safe user fields.
POST	/api/auth/login	Public	Authenticate; returns session/token according to implementation.
POST	/api/auth/logout	Authenticated	Invalidate session/token where supported.
POST	/api/complaints	Citizen	Create complaint; returns 201 and complaint ID.
GET	/api/complaints/mine	Citizen	List only current citizen’s complaints.
GET	/api/complaints/:id	Owner or admin	Retrieve one complaint subject to ownership/role checks.
GET	/api/admin/complaints	Admin	List and filter all complaints.
PATCH	/api/admin/complaints/:id	Admin	Update allowed status, priority, or assignment.
GET	/api/admin/dashboard	Admin	Return aggregate complaint counts.

3.4 Example complaint request and response

Request body example:

```
{"category":"Signal malfunction","description":"Traffic signal is not working","locationDescription":"Main Road near the bus stop"}
```

Successful response (example):

```
{"message":"Complaint created","complaint":{"id":"<generated-
id>","status":"Submitted"}}
```

3.5 Error handling

HTTP status	Use
400 Bad Request	Missing fields, invalid status/priority, or malformed request.
401 Unauthorized	Missing or invalid authentication.
403 Forbidden	Authenticated user lacks required role or permission.
404 Not Found	Complaint identifier does not exist or is not available to the requester, according to privacy policy.
409 Conflict	Duplicate email or conflicting resource state.
500 Internal Server Error	Unexpected failure; response must not reveal internal stack traces or secrets.

4. Deployment and Maintenance

- Development environment: separate frontend, API, and database configuration.
- Store connection strings and signing secrets in environment variables, not source control.
- Use a test database and seeded test accounts for test execution.
- Review changes through pull requests and run automated checks before merging.
- Document any differences between this baseline design and the delivered implementation.